Pavan Kumar Medanki – Frontend Developer

pavankumarmedanki260@gmail.com
+91-8688956064
portfolio
linkedin.com/in/pavan-medanki
github.com/pavanmedanki260

Professional Summary:

Frontend Engineer with 4+ years building high-scale cloud FinOps and observability platforms using React and TypeScript. Architected a DB-driven config rendering engine that eliminated frontend deployments for new dashboard modules — cutting onboarding from 5 days to 4 hours. Deep experience in state management strategy, monorepo architecture (Nx/Turborepo), large-scale data visualization (500K–1M+ records), and automation-first frontend systems that directly improve engineering velocity while integrating frontend applications with backend services and databases.

Technical Skills:

Languages & Frameworks: TypeScript, JavaScript (ES6+), React.js,Node.js,Express.js, HTML5, CSS3, SCSS.
State Management: Redux (Toolkit), Recoil, Context API, React Query (TanStack Query).
AI-Assisted Development: GitHub Copilot, Copilot Skills, Copilot Agents — service provisioning form automation.
Data Visualization: ECharts, D3.js, Chart.js — optimized for 500K–1M+ record datasets.
Frontend Architecture: Config-Driven UI, Micro-Frontends, Monorepo (Nx, Turborepo), Component Libraries.
UI & Design Systems: Tailwind CSS, Mantine UI, Material UI, Responsive Design, WCAG Accessibility (a11y).
Performance: Code Splitting, Lazy Loading, Memoization, Virtualized Rendering, Lighthouse Audits.
API & Real-Time: REST APIs, WebSockets, Dynamic API Orchestration, Async Data Handling.
Testing & Quality: Jest, React Testing Library, Code Reviews, Agile/Scrum.
Cloud & FinOps: AWS (S3, CloudFront, Lambda), Azure, GCP, Cost Allocation, Chargeback/Showback.
DevOps & CI/CD: Git, GitHub Actions, Docker, Jenkins, AWS CodePipeline.
Database: PostgreSQL.

Education:

- **B.Tech — Electrical and Electronics Engineering 2018 – 2021**
- Gudlavalleru Engineering College, Andhra Pradesh

Professional Experience:

EverestDX | Hyderabad **Oct 2022 – Present**
Frontend Engineer
Project 1: Multi-Cloud Cost Management Platform (React · TypeScript · Context API · Nx Monorepo)

- Architected a DB-driven UI rendering engine that decouples frontend from feature releases — API endpoints, widget configs, filters, and chart types resolved at runtime per route, eliminating code deployments for new dashboard modules entirely.
- Reduced new module onboarding from 3–5 days to under 4 hours (80% faster) by replacing hardcoded module logic with a fully config-driven rendering system across 14+ resource modules.
- Managed application state via Context API — scoped providers per dashboard module to prevent stale cost data propagation and eliminate unnecessary re-renders on high-frequency WebSocket-updated screens.
- Engineered a route-based dynamic API orchestration layer that resolves and fires 5–10 parallel API calls per module from config — enabling new cloud services to be integrated with zero frontend code changes.
- Optimized rendering pipeline for 500K–1M+ record datasets using ECharts canvas rendering and windowed table virtualization, sustaining sub-2s interactions on data-heavy cost reports.
- Built 20+ reusable UI components (KPI cards, filter bars, chart wrappers, data tables) adopted across 14+ modules, cutting duplicate development effort by ~65% and enforcing visual consistency platform-wide.
- Enforced module boundaries in Nx monorepo across 3 product verticals, preventing cross-domain imports and reducing unintended coupling between FinOps, observability, and marketplace modules.
- Delivered real-time cost anomaly detection UI via WebSocket integration — dashboards reflect live cloud spend changes across AWS and Azure without manual refresh, used daily by 30–50 internal teams.

Project 2: Cloud Marketplace Platform (React · TypeScript · Recoil · Redux · GitHub Copilot Skills)

- Identified that 80%+ of service catalog development was repetitive boilerplate — engineered a GitHub Copilot Skills and Agents system that auto-produces a complete frontend vertical (component tree, Recoil store, TypeScript types, API integration layer) from a single service schema, with agents orchestrating multi-step provisioning form generation end-to-end.
- Separated server-cache state (Recoil) from shared UI state (Redux) across service catalog modules — preventing stale provisioning state between multi-step workflows and enabling predictable state transitions across complex infrastructure forms.
- Cut service catalog creation from 1–2 days to under 2 hours (85–90% faster) by eliminating manual scaffolding — enabling non-frontend engineers to onboard new services with zero architectural decisions.
- Standardized frontend architecture across all service catalogs by generating from a single opinionated template, making every catalog structurally identical and immediately maintainable by any team member.
- Built dynamic config-driven provisioning forms that adapt field structure, validation rules, and API payloads based on service type — one UI shell supporting 10+ infrastructure service categories.
- Integrated frontend provisioning UI with backend pipeline APIs supporting real-time service request status tracking across multi-step infrastructure workflows.